UTS

PEMBELAJARAN MESIN



NAMA : Adzkia Nayla Anshari

NIM : 202531139

KELAS : H

DOSEN : Bu Rizqia Cahyaningtyas, S.T., M.Kom

NO.PC : 10

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I 4
PENDAHULUAN 4
1.1 Rumusan Masalah 4
1.2 Tujuan Masalah 4
1.3 Manfaat Masalah 4
BAB II 5
LANDASAN TEORI 5
2.1 Pembelajaran Mesin 5
2.2 Supervised Learning 5
2.3 Dataset Bank Customer 5
2.4 Exploratory Data Analysis (EDA) 5
2.5 Prepocessing Data 6
2.6 Algoritma Decision Tree 6
2.7 Regresi Linear 6
2.8 Evaluasi Model Klasifikasi dan Regresi 7
2.8.1 Klasifikasi 7
2.8.2 Regresi 7
BAB III 8
HASIL 8
3.1 Dataset 8
3.2 Penjelasan Kode 8
3.2.1 Import Library 8
3.2.2 Membaca Dataset 8
3.2.3 Memahami Isi Dataset 9
3.2.4 Periksa Data Kosong dan Data Duplikat 10
3.2.5 Lihat Jumlah Data Pada Terget 10
3.2.6 Distribusi Usia 11
3.2.7 Boxplot Saldo Nasabah 12
3.2.8 Heatmap Korelasi 12
3.2.9 Menentukan Fitur dan Target 13

3.2.10 Mengubah Data Kategori Menjadi Angka 14
3.2.11 Membagi Data Training dan Testing 14
3.2.12 Membuat dan Melatih Model Decision Tree 15
3.2.13 Prediksi Decision Tree 15
3.2.14 Accuracy Decision Tree 16
3.2.15 Confusion Matrix 16
3.2.16 Precision, Recall, dan F1-Score 17
3.2.17 Visualisasi Decision Tree 18
3.2.18 Feature Importance 19
3.2.19 Menentukan Fitur dan Target Regresi Linear 20
3.2.20 Membagi Data Regresi 20
3.2.21 Membuat dan Melatih Model Regresi Linear 21
3.2.22 Prediksi Regresi Linear 21
3.2.23 Evaluasi Regresi 22
3.2.24 Grafik Aktual dan Prediksi Regresi 22
3.3 Ringkasan Hasil Decision Tree 23
3.4 Ringkasan Hasil Regresi Linear 23
BAB IV 24
PENUTUP 24
4.1 Kesimpulan 24
DAFTAR PUSTAKA 25

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan praktik UTS pada Pembelajaran Mesin, permasalahan yang diangkat adalah cara memahami sifat dataset nasabah bank melalui EDA, bagaimana cara melakukan pemeriksaan dan eksplorasi data sebelum proses pemodelan, dan juga cara mempersiapkan data dengan atribut kategori serta numerik. Praktikum ini juga menjelaskan penerapan algoritma decision tree untuk mengklasifikasikan nasabah ke dalam kategori yes atau no serta regresi linear untuk memperkirakan nilai saldo. Selain itu, penting untuk mengetahui seberapa efektif kinerja kedua model tersebut berdasarkan metrik evaluasi yang relevan.

1.2 Tujuan Masalah

Tujuan dari praktik ini adalah untuk memahami struktur dataset, memeriksa keberadaan nilai kosong dan data yang berulang, melaksanakan analisis eksplorasi data (EDA) menggunakan tabel statistik dan grafik, memisahkan fitur dari target, membagi dataset menjadi data latih dan data uji, melatih model decision tree dan regresi linear serta menilai hasil prediksi dengan menggunakan metrik untuk klasifikasi dan regresi.

1.3 Manfaat Masalah

Praktikum ini memberikan pengalaman dalam penggunaan Python, JupyterLab, Pandas, Matplotlib, dan Scikit-learn untuk memproses dataset. Selain itu, praktikum ini juga membantu memahami perbedaan antara klasifikasi dan regresi, memahami alasan dibutuhkannya preprocessing, membaca visualisasi dari decision tree, serta mengevaluasi model berdasarkan hasil pengujian dan tidak hanya berdasarkan proses pelatihan.

BAB II
LANDASAN TEORI

2.1 Pembelajaran Mesin

Pembelajaran mesin merupakan cabang dari kecerdasan buatan yang mempelajari pola dalam data, sehingga memungkinkan komputer untuk membuat prediksi atau mengambil keputusan. Dalam praktikum ini, model tidak dikembangkan dengan menetapkan semua aturan keputusan secara manual. Algoritma akan menerima data pelatihan untuk memahami hubungan antara fitur (variabel masukan) dan target (variabel yang ingin diprediksi). Setelah model dilatih akan dilakukan pengujian menggunakan data testing yang tidak terlibat saat fase pelatihan. Pembelajaran ini juga menjelaskan perbedaan antara supervised learning dan unsupervised learning. Supervised learning menggunakan data yang memiliki label atau target, sementara unsupervised learning menemukan pola dalam data tanpa adanya label target. Pada praktikum ini termasuk dalam supervised learing karena dataset memiliki kolom target y, dan model belajar dari contoh yang sudah punya jawaban.

2.2 Supervised Learning

Supervised learning merupakan metode dalam pembelajaran mesin yang memanfaatkan data berlabel atau memiliki target sebagai dasar pelatihan model. Dalam eksperimen ini, metode ini diterapkan melalui dataset Bank Customers, dengan kolom ‘y’ sebagai klasifikasi dan ‘balance’ sebagai regresi. Model dilatih menggunakan data pelatihan untuk memahami pola, kemudian diuji dengan menggunakan data testing untuk mengevaluasi kemampuannya dalam melakukan prediksi.

2.3 Dataset Bank Customer

Dataset yang digunakan bernama 04_bank_customers.csv dan terdiri dari 45,211 baris serta 17 kolom. Dataset ini memuat informasi nasabah dan aktivitas kampanye pemasaran bank. Kolom atribut kategori antara lain job, marital, education, default, housing, loan, contact, month, dan poutcome. Kolom atribut numerik antara lain age, balance, day, duration, campaign, pdays, dan previous. . Kolom y merupakan target yang menunjukkan apakah nasabah berlangganan deposito berjangka, dengan nilai yes atau no. Dalam eksperimen regresi, balance dipilih sebagai target numerik.

2.4 Exploratory Data Analysis (EDA)

EDA merupakan langkah pertama untuk mengenali isi dan ciri khas data sebelum membuat model. EDA bisa dilakukan dengan mengeksplorasi ukuran dataset, tipe data, statistik deskriptif, nilai kosong, data duplikat, distribusi target, histogram, boxplot, dan korelasi. Fokusnya adalah untuk menemukan pola-pola yang muncul, memahami jangkauan data, mengenali ketidakseimbangan antar kelas, serta mengidentifikasi kemungkinan nilai ekstrem. EDA secara mandiri tidak dapat membuktikan adanya hubungan sebab-akibat.

Pada praktikum ini, visualisasi distribusi variabel target dilakukan dengan diagram batang, usia dengan histogram, keseimbangan dengan boxplot, serta hubungan antara variabel numerik menggunakan heatmap korelasi. Temuan EDA memberikan panduan tentang hal-hal yang patut diperhatikan dalam tahap preprocessing dan evaluasi.

2.5 Prepocessing Data

Prepocessing adalah langkah yang diperlukan untuk mempersiapkan data agar dapat digunakan oleh algoritma. Proses yang dilakukan dalam latihan ini mencakup pengecekan data yang hilang, pemeriksaan untuk nilai duplikat, pemisahan antara fitur dan target, mengonversi variabel kategorikal ke dalam angka, dan juga memecah data menjadi set pelatihan dan set pengujian. Setiap algoritma memerlukan jenis input yang spesifik, sehingga data harus diatur sesuai dengan kebutuhan.

Pada praktikum, One-Hot Encoding diterapkan pada kolom yang bersifat kategorikal. Metode ini mengubah kolom indikator kategori menjadi beberapa kolom indikator bernilai 0 atau 1 untuk setiap kategori yang ditemukan yang memungkinkan data teks untuk diubah menjadi format numerik tanpa menciptakan urutan angka yang ada maknanya. Cara ini tidak memberi urutan palsu pada kategori. Proporsi pembagian data adalah 80:20, di mana 80% digunakan untuk pelatihan dan 20% untuk pengujian. Pengaturan parameter stratify=y dalam pengklasifikasian bertujuan untuk mempertahankan keseimbangan proporsi kelas di kedua bagian data tersebut.

2.6 Algoritma Decision Tree

Decision tree meupakan algoritma supervised learning yang digunakan untuk klasifikasi maupun regresi. Model ini memberikan struktur menyerupai pohon yang terdiri dari akar, cabang, dan daun. Setiap pembagian memisahkan data berdasarkan kondisi dari fitur tertentu, sementara daun memberikan hasil akhir. Struktur pohon terdiri dari root node (akar), decision node (keputusan), branch (cabang), dan leaf node (daun). Root adalah titik awal bagi pemisahan data. Setiap node melakukan pengujian terhadap fitur spesifik, cabang menunjukkan hasil uji, dan leaf mengeluarkan keputusan akhir atau kelas prediksi.

Salah satu keuntungan dari decision tree ini adalah aturan yang dihasilkan cukup mudah untuk dipahami dan dapat divisualisasikan. Sisi negatifnya, pohon bisa menjadi sangat rumit dan mengalami overfitting, yaitu penyesuaian berlebih terhadap pola data pelatihan yang mengakibatkan penurunan kemampuan pada data yang belum pernah dilihat sebelumnya. Namun, kedalaman pohon yang berlebihan dapat menyebabkan terlalu menyesuaikan dengan data latih (overfitting). Pembatasan max_depth dalam program adalah salah satu cara untuk mengendalikan pertumbuhan pohon, sehingga kedalaman pohon dapat dibatasi.

2.7 Regresi Linear

Regresi Linear merupakan metode untuk memperkirakan nilai target yang berbentuk angka berdasarkan satu atau lebih variabel masukan. Regresi linear sederhana menggunakan satu fitur, sedangkan regresi linear berganda menggunakan beberapa fitur. Secara umum hubungan model dapat ditulis dengan

$$y = b0 + b1x1 + b2x2 + ... + bnxn.$$

$\hat{y}$ merupakan nilai prediksi, b_0 merupakan konstanta, b_1 sampai b_n merupakan koefisien, dan x_1 sampai x_n merupakan fitur masukan. Koefisien menunjukkan arah dan besarnya hubungan linear antara fitur dengan target ketika fitur lain dianggap tetap. Model dilatih dengan mencari koefisien yang meminimalkan jumlah kuadrat kesalahan. Dalam praktikum ini balance dipilih sebagai target numerik dan enam kolom numerik digunakan sebagai fitur.

2.8 Evaluasi Model Klasifikasi dan Regresi

2.8.1 Klasifikasi

Untuk klasifikasi, accuracy adalah perbandingan antara jumlah prediksi yang benar dan total data pengujian. Confussion matrix mempresentasikan perbandingan antara kelas yang sebenarnya dan yang diprediksi. Precision mengukur ketepatan dari prediksi suatu kategori, sedangkan recall menentukan seberapa banyak anggota dari kategori yang dapat ditemukan, dan F1-score merupakan kombinasi dari precision dan recall. Dalam situasi di mana jumlah kelas tidak seimbang, akurasi harus dibaca bersamaan dengan metrik lain. Dalam kasus data yang tidak seimbang, akurasi dapat memberikan gambaran yang belum holistik. Oleh karena itu precision, recall, F1-score, dan confussion matrix perlu dianalisis bersama-sama. Ini sangat penting untuk dataset karena jumlah kelas 'no' jauh lebih dominan dibandingkan dengan kelas 'yes'.

2.8.2 Regresi

Untuk analisis regresi, penilaian tidak melibatkan accuracy seperti dalam klasifikasi. R-squared (R^2) menggambarkan seberapa besar variasi dari target dapat dijelaskan oleh model dalam data evaluasi. Semakin dekat nilai R^2 ke satu, semakin baik model dalam menjelaskan variasi dari target, sebaliknya, nilai yang mendekati nol menunjukkan kemampuan penjelasan yang lebih rendah daripada rata-rata baseline. R^2 bisa saja bernilai negatif jika model tersebut lebih buruk dibandingkan dengan prediksi menggunakan rata-rata dari target. Mean Absolute Error (MAE) yaitu mengukur rata-rata dari nilai absolut perbedaan antara prediksi dan nilai sebenarnya. Root Mean Squared Error (RMSE) adalah akar dari rata-rata kuadrat kesalahan. Chicco, Warrens, dan Jurman (2021) membahas penggunaan R^2 dan metrik kesalahan seperti MAE dan RMSE dalam evaluasi regresi. Metrik-metrik tersebut sebaiknya dibaca bersama karena masing-masing memberi informasi yang berbeda.

BAB III
HASIL

3.1 Dataset

Praktikum dilakukan menggunakan Python pada JupyterLab. Notebook terdiri dari 24 cell kode yang dijalankan secara berurutan. Library utama yang digunakan adalah Pandas untuk membaca dan mengolah tabel, Matplotlib untuk visualisasi, serta Scikit-learn untuk preprocessing, pembagian data, pelatihan model, dan evaluasi. Dataset yang digunakan adalah 04_bank_customers.csv dengan pemisah kolom titik koma. . Hasil pemeriksaan menunjukkan dataset mempunyai 45.211 baris dan 17 kolom. Data terdiri dari atribut demografi, kondisi keuangan, dan riwayat kontak kampanye pemasaran. Kolom y menjadi target klasifikasi, sedangkan balance dipilih sebagai target regresi.

3.2 Penjelasan Kode

3.2.1 Import Library

```
import pandas as pd
from IPython.display import display
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LinearRegression

from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

202531139

Cell pertama adalah import library, library adalah kumpulan perintah yang membantu kita mengolah data. Cell ini memanggil library yang akan digunakan selama praktikum. Pandas diberi nama singkat pd untuk membaca dan mengolah data berbentuk tabel. display digunakan agar tabel hasil dapat ditampilkan lebih rapi di JupyterLab. Matplotlib.pyplot diberi nama plt untuk membuat grafik. Scikit-learn digunakan untuk membagi data, membuat model, dan menghitung hasil evaluasi. train_test_split membagi dataset menjadi data training dan testing. ColumnTransformer mengatur perlakuan berbeda untuk kelompok kolom. OneHotEncoder mengubah kategori menjadi kolom angka indikator. Pipeline menggabungkan preprocessing dan algoritma dalam satu alur. DecisionTreeClassifier adalah model klasifikasi pohon keputusan, sedangkan plot_tree menampilkan bentuk pohonnya. LinearRegression adalah model regresi linear. Fungsi metrics digunakan untuk mengukur hasil klasifikasi dan regresi.

3.2.2 Membaca Dataset

```
data = pd.read_csv('04_bank_customers.csv', sep=';')

print('Ukuran dataset:', data.shape)
data.head()
```

202531139

Perintah pd.read_csv() membaca file CSV bernama 04_bank_customers.csv. Parameter sep=';' berarti pemisah antar kolom adalah titik koma. Jika pemisah tidak ditulis dengan benar, data bisa terbaca sebagai satu kolom panjang. data.shape menampilkan jumlah baris dan kolom. Hasilnya adalah 45.211 baris dan 17 kolom. data.head() menampilkan lima baris pertama untuk memastikan file berhasil dibaca dan memberi gambaran isi dataset. Pemeriksaan awal ini memastikan file terbaca dengan benar dan kolom tidak tergabung menjadi satu.

Output:

```
Ukuran dataset: (45211, 17)
```

202531139

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pda		y
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no

3.2.3 Memahami Isi Dataset

```
print('Informasi dataset:')
data.info()

print('Statistik data angka:')
display(data.describe())

print('Nama seluruh kolom:')
print(data.columns)
```

202531139

Cell ini digunakan untuk memahami struktur dataset sebelum menentukan fitur dan target. data.info() menampilkan nama kolom, tipe data, dan jumlah nilai yang tidak kosong. data.describe() menampilkan ringkasan statistik kolom numerik, seperti jumlah data, rata-rata, standar deviasi, nilai minimum, kuartil, dan maksimum. display() membantu menampilkan tabel statistik. data.columns menampilkan seluruh nama kolom.

Output:

```
Informasi dataset:
<class 'pandas.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  str
 2   marital    45211 non-null  str
 3   education  45211 non-null  str
 4   default    45211 non-null  str
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  str
 7   loan       45211 non-null  str
 8   contact    45211 non-null  str
 9   day        45211 non-null  int64
 10  month      45211 non-null  str
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-n
 16  y          45211 non-n
dtypes: int64(7), str(10)
memory usage: 8.1 MB
```

202531139

```
Statistik data angka:
```

	age	balance	day	duration	campaign	pdays	previous
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000
mean	40.936210	1362.272058	15.806419	258.163080	2.763841	40.197828	0.580323
std	10.618762	3044.765829	8.322476	257.527812	3.098021	100.128746	2.303441
min	18.000000	-8019.000000	1.000000	0.000000	1.000000	-1.000000	0.000000
25%	33.000000	72.000000	8.000000	103.000000	1.000000	-1.000000	0.000000
50%	39.000000	448.000000	16.000000	180.000000	2.000000	-1.000000	0.000000
75%	48.000000	1428.000000	21.000000	319.000000	3.000000	-1.000000	0.000000
max	95.000000	102127.000000	31.000000	4918.000000	63.000000	871.000000	275.000000

```
Nama seluruh kolom:
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'y'],
      dtype='str')
```

3.2.4 Periksa Data Kosong dan Data Duplikat

```
print('Jumlah data kosong setiap kolom:')
print(data.isnull().sum())

print('Jumlah baris duplikat:')
print(data.duplicated().sum())
```

202531139

data.isnull().sum() menghitung jumlah nilai kosong pada setiap kolom. data.duplicated().sum() menghitung baris yang sama persis dengan baris sebelumnya. Hasil pemeriksaan pada dataset ini menunjukkan tidak ada nilai kosong dan tidak ada baris duplikat. Karena itu, program tidak melakukan pengisian nilai kosong atau penghapusan baris duplikat. Pemeriksaan tetap penting karena dataset lain bisa memiliki masalah tersebut.

Output:

```
Jumlah data kosong setiap kolom:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
Jumlah baris duplikat:
0
```

202531139

3.2.5 Lihat Jumlah Data Pada Terget

```
print(data['y'].value_counts())

data['y'].value_counts().plot(kind='bar')
plt.title('Jumlah Nasabah Berdasarkan Target')
plt.xlabel('Target y')
plt.ylabel('Jumlah Nasabah')
plt.xticks(rotation=0)
plt.show()
```

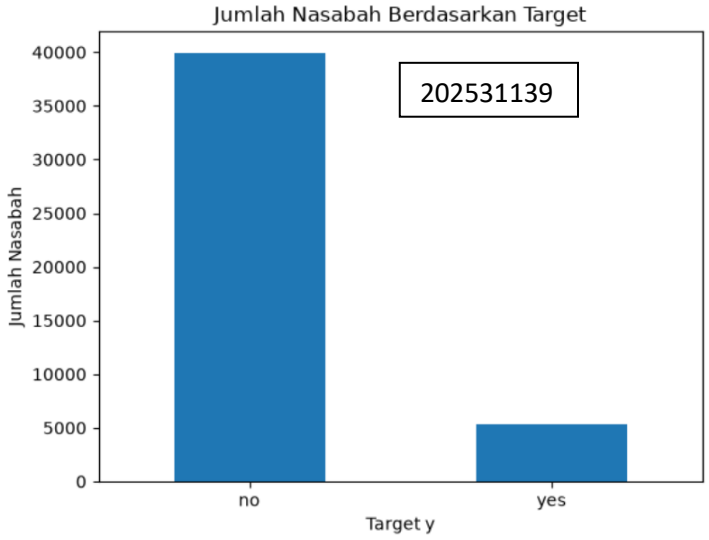
202531139

Kolom y merupakan target yang akan diprediksi oleh decision tree. data['y'].value_counts() menghitung banyaknya data pada setiap kategori target. Grafik batang dibuat dari hasil perhitungan tersebut. Sumbu mendatar menunjukkan kelas no dan yes, sedangkan sumbu tegak menunjukkan jumlah data. Nilai yes berarti nasabah berlangganan deposito, sedangkan no berarti tidak. Hasilnya adalah 39.922 kelas no dan 5.289 kelas yes. Hal ini menunjukkan kalau target tidak seimbang karena kelas no jauh lebih banyak. Kondisi ini perlu diperhatikan saat membaca accuracy karena model bisa

terlihat bagus dengan lebih sering memprediksi kelas mayoritas. Oleh karena itu, hasil Decision Tree tidak hanya dilihat dari accuracy, tetapi juga precision, recall, F1-score, dan confusion matrix.

Output:

```
y
no     39922
yes     5289
Name: count, dtype: int64
```



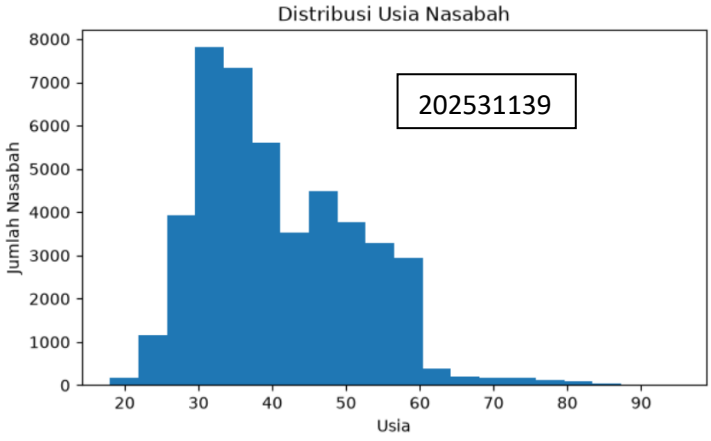
3.2.6 Distribusi Usia

```
plt.figure(figsize=(7, 4))
plt.hist(data['age'], bins=20)
plt.title('Distribusi Usia Nasabah')
plt.xlabel('Usia')
plt.ylabel('Jumlah Nasabah')
plt.show()
```

202531139

plt.figure() mengatur ukuran area gambar. plt.hist(data['age'], bins=20) membuat histogram dari kolom usia dengan 20 kelompok rentang. Judul dan label sumbu ditambahkan agar grafik mudah dipahami. Histogram menunjukkan bagaimana usia nasabah tersebar dan pada rentang usia mana jumlah nasabah lebih banyak.

Output:



Grafik menunjukkan sebagian besar data berada pada rentang usia dewasa, sedangkan jumlah nasabah pada usia sangat tinggi lebih sedikit. Grafik ini membantu memahami rentang nilai dan pola distribusi sebelum model dibuat.

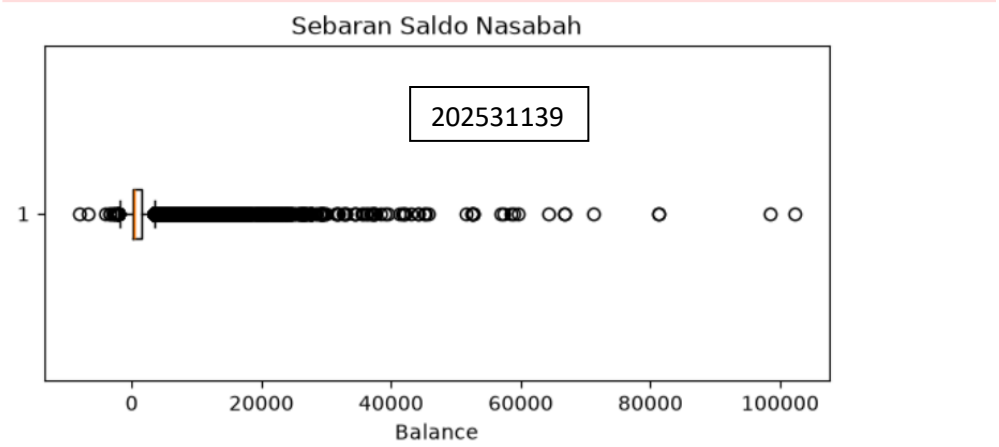
3.2.7 Boxplot Saldo Nasabah

```
]: plt.figure(figsize=(7, 3))
   plt.boxplot(data['balance'], vert=False)
   plt.title('Sebaran Saldo Nasabah')
   plt.xlabel('Balance')
   plt.show()
```

202531139

Boxplot digunakan untuk melihat sebaran nilai saldo (balance) dan membantu melihat apakah ada nilai yang jauh berbeda dari sebagian besar data. plt.boxplot(data['balance'], vert=False) membuat boxplot horizontal dari kolom balance. Boxplot menampilkan median, kuartil, rentang data, dan titik yang dapat terdeteksi sebagai pencilan. Pada data saldo, rentang nilai cukup lebar dan terdapat nilai yang jauh dari sebagian besar data. Hal ini perlu diperhatikan karena nilai ekstrem dapat memengaruhi model regresi.

Output:



3.2.8 Heatmap Korelasi

```
data_angka = data.select_dtypes(include=['int64', 'float64'])
korelasi = data_angka.corr()

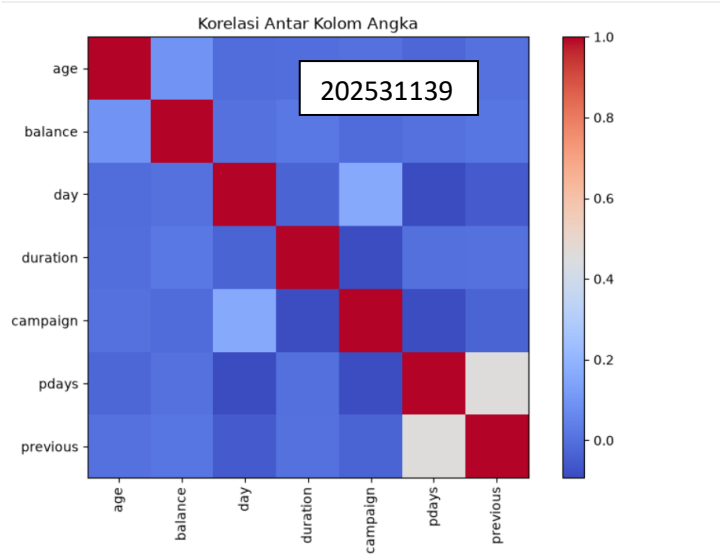
plt.figure(figsize=(8, 6))
plt.imshow(korelasi, cmap='coolwarm')
plt.colorbar()
plt.xticks(range(len(korelasi.columns)), korelasi.columns, rotation=90)
plt.yticks(range(len(korelasi.columns)), korelasi.columns)
plt.title('Korelasi Antar Kolom Angka')
plt.tight_layout()
plt.show()
```

202531139

data.select_dtypes(include=['int64','float64']) memilih kolom dengan tipe angka. corr() menghitung korelasi antar kolom angka. imshow() menampilkan matriks korelasi sebagai heatmap, sedangkan colorbar memberi skala warna. Label kolom ditampilkan pada kedua sumbu. Korelasi positif menunjukkan dua variabel cenderung bergerak searah, korelasi negatif menunjukkan kecenderungan

berlawanan, dan nilai mendekati nol menunjukkan hubungan linear yang lemah. Korelasi tidak membuktikan sebab-akibat.

Output:



3.2.9 Menentukan Fitur dan Target

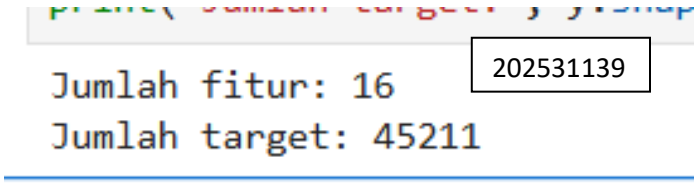
```
X = data.drop('y', axis=1)
y = data['y']

print('Jumlah fitur:', X.shape[1])
print('Jumlah target:', y.shape[0])
```

202531139

Untuk Decision Tree, kolom y digunakan sebagai target yang ingin diprediksi. Kolom selain y digunakan sebagai fitur atau informasi yang membantu model membuat prediksi. X berisi semua kolom selain y karena y adalah label yang ingin diprediksi. data.drop('y', axis=1) menghapus kolom y dari tabel fitur; axis=1 menandakan penghapusan kolom. Variabel y menyimpan target klasifikasi. X.shape[1] menampilkan jumlah fitur, yaitu 16, sedangkan y.shape[0] menampilkan jumlah label, yaitu 45.211. Pemisahan ini mencegah target ikut menjadi masukan model.

Output:



3.2.10 Mengubah Data Kategori Menjadi Angka

```
kolom_kategori = X.select_dtypes(include=['object']).columns
kolom_angka = X.select_dtypes(exclude=['object']).columns

preprocessing = ColumnTransformer([
    ('kategori', OneHotEncoder(handle_unknown='ignore'), kolom_kategori),
    ('angka', 'passthrough', kolom_angka)
])

print('Kolom kategori:', list(kolom_kategori))
print('Kolom angka:', list(kolom_angka))
```

202531139

Model machine learning membutuhkan masukan berbentuk angka. Beberapa kolom dataset berisi kategori atau teks, misalnya job, marital, dan education. One-Hot Encoding mengubah kategori menjadi kolom angka 0 dan 1. Kolom angka tidak perlu diubah pada tahap ini, sehingga diteruskan menggunakan passthrough. select_dtypes(include=['object']) memilih kolom bertipe object yang pada dataset ini berisi kategori/teks. select_dtypes(exclude=['object']) memilih kolom selain object, yaitu kolom numerik. Hasilnya ada sembilan kolom kategori: job, marital, education, default, housing, loan, contact, month, dan poutcome; serta tujuh kolom angka: age, balance, day, duration, campaign, pdays, dan previous. ColumnTransformer kemudian membuat dua perlakuan: OneHotEncoder diterapkan pada kolom kategori, sedangkan 'passthrough' mempertahankan kolom angka apa adanya. handle_unknown='ignore' mencegah kategori baru pada data uji menyebabkan proses encoding gagal.

Output:

```
Kolom kategori: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']
Kolom angka: ['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
```

202531139

3.2.11 Membagi Data Training dan Testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print('Jumlah data training:', len(X_train))
print('Jumlah data testing:', len(X_test))
```

202531139

Data dibagi menjadi 80% data training dan 20% data testing. Data training digunakan untuk mengajari model, sedangkan data testing digunakan untuk menguji model. train_test_split membagi X dan y menjadi data training dan testing. test_size=0.2 berarti 20% data digunakan untuk pengujian dan 80% untuk pelatihan. random_state=42 membuat pembagian dapat diulang dengan hasil yang sama. stratify=y menjaga perbandingan kelas no dan yes pada data training dan testing agar mendekati distribusi awal. Hasil pembagian adalah 36.168 baris training dan 9.043 baris testing.

Output:

```
Jumlah data training: 36168
Jumlah data testing: 9043
```

202531139

3.2.12 Membuat dan Melatih Model Decision Tree

```
model_dt = Pipeline([
    ('preprocessing', preprocessing),
    ('model', DecisionTreeClassifier(max_depth=6, random_state=42))
])

model_dt.fit(X_train, y_train)

print('Model Decision Tree selesai dilatih.')
```

202531139

Pipeline menyatukan tahap preprocessing dan model. Tahap pertama menjalankan ColumnTransformer, lalu tahap kedua menjalankan DecisionTreeClassifier. max_depth=6 membatasi kedalaman maksimum pohon agar model tidak tumbuh terlalu kompleks. random_state=42 menjaga hasil acak internal agar dapat direproduksi. model_dt.fit(X_train, y_train) melatih seluruh alur menggunakan data training. Karena preprocessing berada di dalam Pipeline, proses One-Hot Encoding dipelajari dari data training dan diterapkan konsisten ke data testing.

Output:

```
Model Decision Tree selesai dilatih.
```

202531139

3.2.13 Prediksi Decision Tree

```
prediksi_dt = model_dt.predict(X_test)

print('Contoh hasil prediksi:')
print(prediksi_dt[:10])

print('Jawaban sebenarnya:')
print(y_test.iloc[:10].values)
```

202531139

Setelah model dilatih, data testing digunakan untuk melihat hasil prediksi. Hasil prediksi kemudian dibandingkan dengan jawaban asli pada y_test. model_dt.predict(X_test) menggunakan model yang sudah dilatih untuk memprediksi kelas pada data testing. Sepuluh prediksi pertama ditampilkan, lalu dibandingkan dengan sepuluh label sebenarnya dari y_test. Perbandingan ini merupakan contoh awal supaya bisa terlihat apakah prediksi sesuai penilaian keseluruhan tetap dilakukan dengan metrik pada cell berikutnya.

Output:

```
Contoh hasil prediksi:
['no' 'no' 'no' 'no' 'no' 'no' 'no' 'yes' 'no' 'no']
Jawaban sebenarnya:
<ArrowStringArray>
['no', 'no', 'no', 'no', 'no', 'no', 'no', 'yes', 'no', 'no']
Length: 10, dtype: str
```

202531139

3.2.14 Accuracy Decision Tree

```
akurasi = accuracy_score(y_test, prediksi_dt)

print('Accuracy Decision Tree:', round(akurasi * 100, 2), '%')
```

202531139

Accuracy menunjukkan perbandingan jumlah prediksi yang benar dengan seluruh data testing. Hasil ditampilkan dalam persen. accuracy_score membandingkan label sebenarnya y_test dengan hasil prediksi prediksi_dt. Hasil 90,11% berarti sekitar 90,11 persen dari 9.043 prediksi sesuai label aktual. Namun, karena jumlah no jauh lebih besar daripada yes, accuracy perlu dibaca bersama confusion matrix dan precision-recall kelas yes.

Output:

```
Accuracy Decision Tree: 90.11 %
```

202531139

3.2.15 Confusion Matrix

```
cm = confusion_matrix(y_test, prediksi_dt, labels=['no', 'yes'])

print('Confusion Matrix:')
print(cm)

plt.figure(figsize=(5, 4))
plt.imshow(cm, cmap='Blues')
plt.title('Confusion Matrix Decision Tree')
plt.xlabel('Hasil Prediksi')
plt.ylabel('Data Sebenarnya')
plt.xticks([0, 1], ['no', 'yes'])
plt.yticks([0, 1], ['no', 'yes'])

for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.colorbar()
plt.show()
```

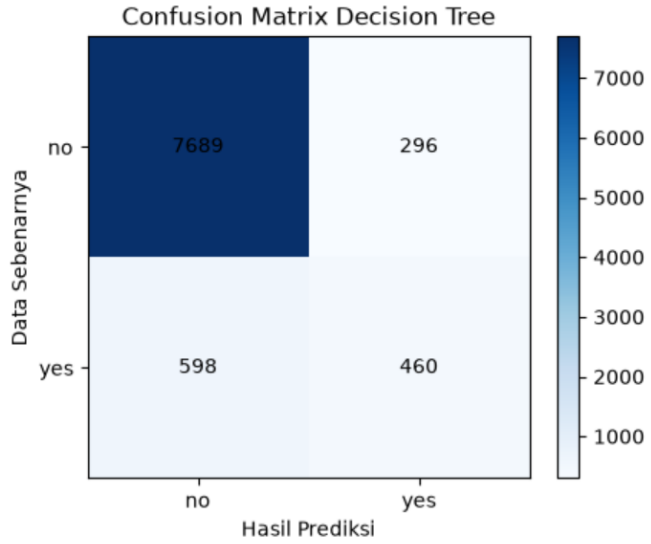
202531139

confusion_matrix menghitung jumlah prediksi benar dan salah untuk setiap kelas. Parameter labels=['no','yes'] menetapkan urutan kelas pada matriks. Baris menunjukkan kelas sebenarnya dan kolom menunjukkan hasil prediksi. Confusion matrix menghasilkan empat kelompok hitungan. Untuk kelas no, 7.689 data no diprediksi no dan 296 data no salah diprediksi yes. Untuk kelas yes, 460 data yes berhasil diprediksi yes, sedangkan 598 data yes salah diprediksi no. Hasil ini memperlihatkan bahwa model lebih baik mengenali kelas no dibandingkan kelas yes. Ketidakseimbangan jumlah kelas menjadi salah satu hal yang perlu diperhatikan ketika membaca hasil. imshow menampilkan angka tersebut dalam bentuk gambar supaya mudah dibaca.

Output:

```
Confusion Matrix:
[[7689  296]
 [ 598  460]]
```

202531139



3.2.16 Precision, Recall, dan F1-Score

```
print(classification_report(y_test, prediksi_dt))
```

202531139

Selain accuracy, kita bisa llihat precision, recall, dan F1-score. classification_report menampilkan precision, recall, F1-score, dan support untuk masing-masing kelas. Precision mengukur: dari seluruh prediksi yang dibuat untuk suatu kategori, berapa yang akurat. Recall mengukur: dari semua data nyata dalam kategori tersebut, berapa yang berhasil diidentifikasi. F1-score adalah rata-rata harmonis antara precision dan recall. Support merujuk pada jumlah data nyata dalam kategori tersebut. Kategori no memiliki precision 0,93 dan recall 0,96 sedangkan kategori yes memiliki precision 0,61 dan recall 0,43. Oleh karena itu, model lebih efektif dalam mengenali no dibandingkan yes.

Output:

202531139	precision	recall	f1-score	support
no	0.93	0.96	0.95	7985
yes	0.61	0.43	0.51	1058
accuracy			0.90	9043
macro avg	0.77	0.70	0.73	9043
weighted avg	0.89	0.90	0.89	9043

3.2.17 Visualisasi Decision Tree

```
#Mengambil model decision tree dari pipeline
pohon = model_dt.named_steps['model']

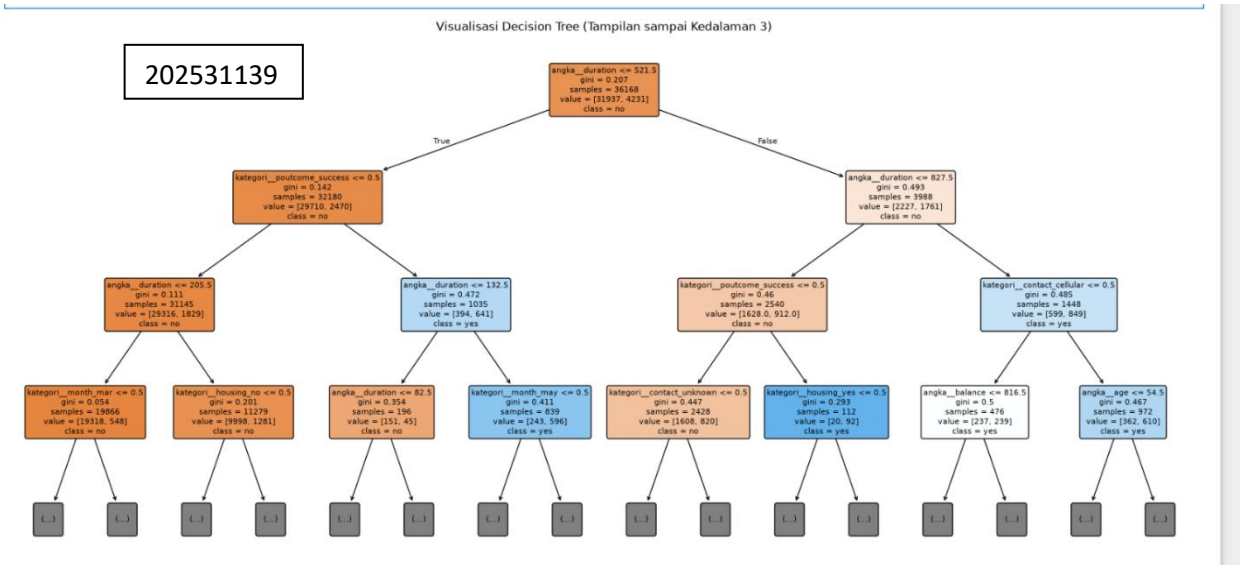
#Mengambil nama fitur setelah proses One-Hot Encoding
nama_fitur = model_dt.named_steps['preprocessing'].get_feature_names_out()

plt.figure(figsize=(22, 10))
plot_tree(
    pohon,
    feature_names=nama_fitur,
    class_names=['no', 'yes'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)
plt.title('Visualisasi Decision Tree (Tampilan sampai Kedalaman 3)')
plt.show()
```

202531139

Bagian ini menampilkan bentuk pohon keputusan yang sudah dipelajari model. model_dt.named_steps['model'] mengambil bagian decision tree dari dalam pipeline yang telah dilatih. get_feature_names_out() mengambil nama fitur setelah One-Hot Encoding. plot_tree menggambar struktur pohon, dengan feature_names untuk nama fitur, class_names untuk label kelas, filled=True untuk memberi warna berdasarkan kelas dominan, dan rounded=True untuk kotak bersudut membulat. max_depth=3 membatasi tampilan gambar sampai kedalaman tiga agar tidak terlalu padat. Pembatasan hanya untuk gambar, bukan mengubah model yang sudah dilatih, model yang dilatih tetap memiliki max_depth=6.

Output:



Cara baca visualisasi pohon:

- Kotak paling atas merupakan akar atau awal pengambilan keputusan.
- Tulisan pada kotak menunjukkan fitur dan kondisi yang digunakan untuk membagi data.
- Gini menunjukkan tingkat campuran kategori pada bagian tersebut.
- Samples menunjukkan jumlah data yang masuk ke bagian tersebut.

- Value menunjukkan jumlah data pada masing-masing kategori.
- Class menunjukkan kategori hasil keputusan pada bagian tersebut.
- Cabang kiri dan kanan menunjukkan hasil dari kondisi yang berbeda.

3.2.18 Feature Importance

```
]: nilai_fitur = pohon.feature_importances_

tabel_fitur = pd.DataFrame({
    'Nama Fitur': nama_fitur,
    'Nilai Pengaruh': nilai_fitur
})

tabel_fitur = tabel_fitur.sort_values(
    by='Nilai Pengaruh',
    ascending=False
)

print(tabel_fitur.head(10))

tabel_fitur.head(10).plot(
    x='Nama Fitur',
    y='Nilai Pengaruh',
    kind='bar',
    legend=False,
    figsize=(9, 4)
)
plt.title('10 Fitur dengan Nilai Pengaruh Terbesar')
plt.xlabel('Nama Fitur')
plt.ylabel('Nilai Pengaruh')
plt.xticks(rotation=60, ha='right')
plt.tight_layout()
plt.show()
```

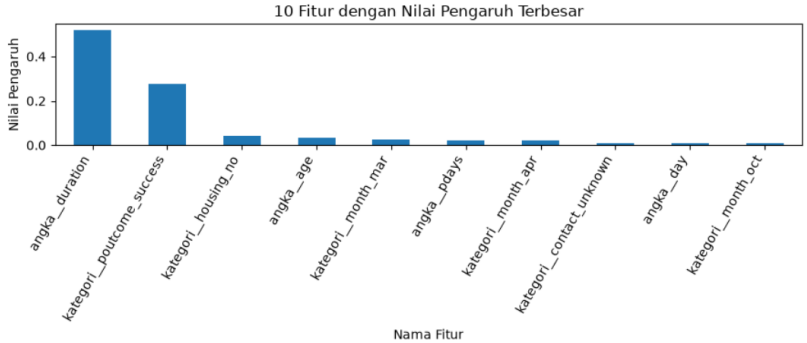
202531139

Decision Tree dapat menampilkan nilai feature importance. Nilai ini menunjukkan seberapa besar suatu fitur membantu model membuat keputusan. pohon.feature_importances_ mengambil nilai kepentingan fitur dari model decision tree. Nilai tersebut disusun dalam DataFrame bersama nama fitur, lalu diurutkan dari nilai terbesar. head(10) mengambil sepuluh fitur teratas. Grafik batang memudahkan melihat fitur yang paling banyak berkontribusi pada pemisahan node menurut kriteria model. Nilai feature importance bukan bukti sebab-akibat dan dapat dipengaruhi struktur pohon serta karakteristik data.

Output:

	Nama Fitur	Nilai Pengaruh
47	angka__duration	0.520528
42	kategori__poutcome_success	0.276027
21	kategori__housing_no	0.041059
44	angka__age	0.031699
35	kategori__month_mar	0.024192
49	angka__pdays	0.022009
28	kategori__month_apr	0.020155
27	kategori__contact_unknown	0.010747
46	angka__day	0.009864
38	kategori__month_oct	0.009451

202531139



3.2.19 Menentukan Fitur dan Target Regresi Linear

```
fitur_regresi = ['age', 'day', 'duration', 'campaign', 'pdays', 'previous']

X_regresi = data[fitur_regresi]
y_regresi = data['balance']

print('Fitur regresi:')
print(X_regresi.head())

print('Target balance:')
print(y_regresi.head())
```

202531139

Cell ini membuat percobaan regresi dengan target balance. fitur_regresi berisi enam kolom masukan: age, day, duration, campaign, pdays, dan previous. X_regresi menyimpan enam fitur tersebut, sedangkan y_regresi menyimpan nilai balance yang akan diperkirakan. head() menampilkan lima baris pertama dari fitur dan target untuk memastikan susunan data sesuai.

Output:

```
Fitur regresi:
   age  day  duration  campaign  pdays  previous
0   58    5       261         1     -1         0
1   44    5       151         1     -1         0
2   33    5        76         1     -1         0
3   47    5        92         1     -1         0
4   33    5       198         1     -1         0
Target balance:
0    2143
1      29
2       2
3    1506
4       1
Name: balance, dtype: int64
```

202531139

3.2.20 Membagi Data Regresi

```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_regresi,
    y_regresi,
    test_size=0.2,
    random_state=42
)

print('Jumlah data training:', len(X_train_reg))
print('Jumlah data testing:', len(X_test_reg))
```

202531139

Data regresi dibagi menjadi 80% training dan 20% testing menggunakan train_test_split. test_size=0.2 menentukan proporsi data uji dan random_state=42 membuat pembagian dapat diulang. Pada regresi tidak digunakan stratify karena target balance berupa angka kontinu, bukan kategori. Hasil pembagian masing-masing 36.168 data training dan 9.043 data testing.

Output:

```
Jumlah data training: 36168
Jumlah data testing: 9043
```

202531139

3.2.21 Membuat dan Melatih Model Regresi Linear

```
model_lr = LinearRegression()

model_lr.fit(X_train_reg, y_train_reg)

print('Model Regresi Linear selesai dilatih.')
```

202531139

LinearRegression() membuat objek model regresi linear. fit(X_train_reg, y_train_reg) melatih model untuk mempelajari hubungan antara enam fitur masukan dan target balance. Model mencari koefisien garis terbaik berdasarkan pendekatan kuadrat terkecil. Pesan print hanya memberi tanda bahwa proses pelatihan selesai

Output:

```
Model Regresi Linear selesai dilatih.
```

202531139

3.2.22 Prediksi Regresi Linear

```
prediksi_lr = model_lr.predict(X_test_reg)

hasil_regresi = pd.DataFrame({
    'Balance Sebenarnya': y_test_reg.values,
    'Balance Prediksi': prediksi_lr
})

hasil_regresi.head(10)
```

202531139

predict(X_test_reg) menghasilkan perkiraan nilai balance untuk setiap baris data uji. Hasilnya digabungkan dengan y_test_reg ke dalam DataFrame agar nilai sebenarnya dan prediksi dapat dibandingkan pada baris yang sama. head(10) menampilkan sepuluh contoh pertama. Perbedaan antara dua kolom tersebut menunjukkan kesalahan prediksi pada masing-masing contoh.

Output:

	Balance Sebenarnya	Balance Prediksi
0	580	1338.432616
1	3644	1473.965407
2	538	932.816679
3	1773	1398.547199
4	217	1761.503315
5	1134	935.010374
6	1085	757.735602
7	127	1184.577003
8	3	1031.940403
9	258	1388.514678

202531139

3.2.23 Evaluasi Regresi

```
nilai_r2 = r2_score(y_test_reg, prediksi_lr)
nilai_mae = mean_absolute_error(y_test_reg, prediksi_lr)
nilai_rmse = mean_squared_error(y_test_reg, prediksi_lr) ** 0.5

print('R-squared (R2):', round(nilai_r2, 4))
print('MAE:', round(nilai_mae, 2))
print('RMSE:', round(nilai_rmse, 2))
```

202531139

Regresi Linear tidak dinilai menggunakan accuracy klasifikasi. R^2 menunjukkan seberapa besar variasi target dapat dijelaskan oleh model. MAE adalah rata-rata selisih absolut antara nilai sebenarnya dan prediksi. RMSE adalah ukuran kesalahan prediksi yang memberi penalti lebih besar pada kesalahan yang besar. Untuk R^2, nilai yang mendekati 1 umumnya lebih baik. Untuk MAE dan RMSE, nilai yang lebih kecil umumnya lebih baik. Hasil percobaan adalah R^2=0,0087, MAE=1.526,63, dan RMSE=3.116,31. R^2 yang mendekati nol menunjukkan fitur yang dipilih hanya menjelaskan bagian sangat kecil dari variasi balance. MAE dan RMSE menunjukkan besar kesalahan dalam satuan balance.

Output:

```
R-squared (R2): 0.0087
MAE: 1526.63
RMSE: 3116.31
```

202531139

3.2.24 Grafik Aktual dan Prediksi Regresi

```
plt.figure(figsize=(7, 5))
plt.scatter(y_test_reg, prediksi_lr, alpha=0.3)
plt.xlabel('Balance Sebenarnya')
plt.ylabel('Balance Prediksi')
plt.title('Perbandingan Balance Sebenarnya dan Prediksi')

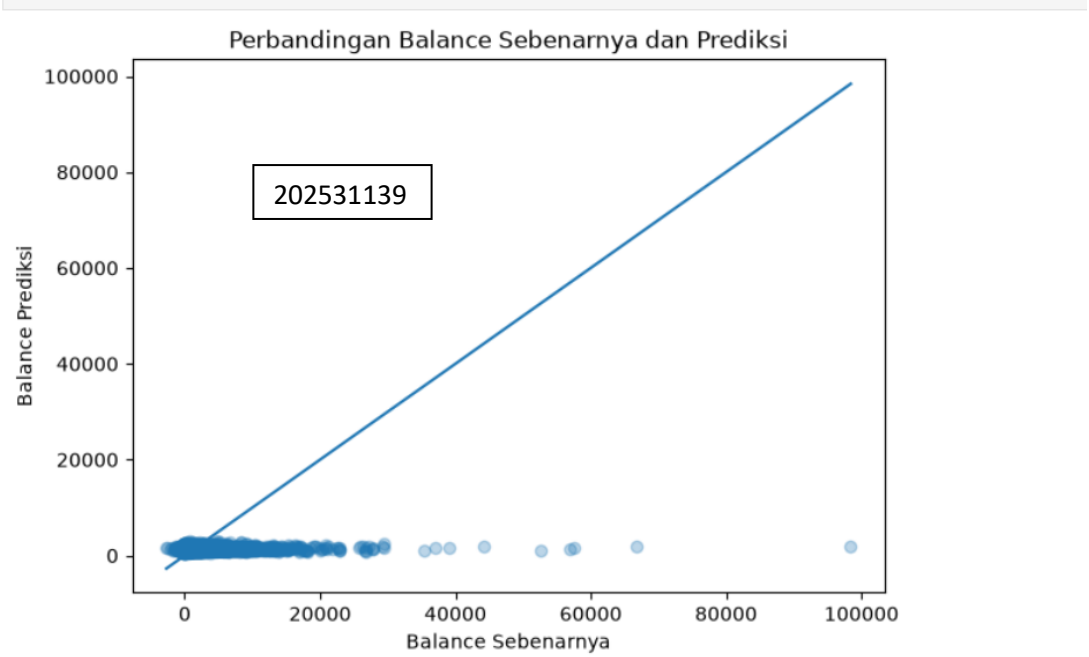
# Garis pembanding
nilai_min = min(y_test_reg.min(), prediksi_lr.min())
nilai_max = max(y_test_reg.max(), prediksi_lr.max())
plt.plot([nilai_min, nilai_max], [nilai_min, nilai_max])

plt.show()
```

202531139

scatter membuat diagram titik dengan sumbu X berisi balance sebenarnya dan sumbu Y berisi balance prediksi. alpha=0.3 membuat titik lebih transparan agar tumpukan titik terlihat. Nilai minimum dan maksimum digunakan untuk menggambar garis diagonal sebagai acuan prediksi sempurna. Jika prediksi tepat, titik akan berada dekat garis diagonal. Sebaran titik yang jauh dari garis menunjukkan prediksi yang meleset. Grafik ini memperkuat hasil evaluasi bahwa model regresi dengan fitur yang dipakai belum baik.

Output:



3.3 Ringkasan Hasil Decision Tree

Data klasifikasi terbagi menjadi 36,168 baris pelatihan dan 9,043 baris pengujian. Model Decision Tree dilatih dengan max_depth=6 dan random_state=42. Hasil pengujian memperoleh accuracy sebesar 90.11%. Confusion matrix dengan urutan label no dan yes menghasilkan [[7689, 296], [598, 460]] yang berarti, 7.689 data no terprediksi sebagai no, 296 data no terprediksi sebagai yes, 598 data yes terprediksi sebagai no, dan 460 data yes terprediksi sebagai yes. Untuk kelas no, precision tercatat 0,93, recall 0,96, dan F1-score 0,95. Sementara itu, untuk kelas yes, precision menunjukkan angka 0,61, recall 0,43, dan F1-score 0,51. Angka recall kelas yes yang sebesar 0,43 mengindikasikan bahwa model hanya berhasil mengidentifikasi sekitar 43% dari seluruh data yang sebenarnya termasuk dalam kelas yes. Oleh karena itu, meskipun akurasi tampak tinggi, model masih mengalami kelemahan dalam mengenali kelas yes.

Visualisasi pohon dibatasi hingga kedalaman tiga untuk memudahkan pembacaan gambar Model yang digunakan untuk prediksi tetap dibangun dengan max_depth=6. Feature importance menunjukkan faktor-faktor yang paling berperan dalam pemisahan node pada pohon. Nilai tersebut adalah ukuran internal dari model dan tidak seharusnya diinterpretasikan sebagai bukti bahwa fitur-fitur itu menjadi penyebab nasabah melakukan deposito.

3.4 Ringkasan Hasil Regresi Linear

Pada regresi, target balance diprediksi menggunakan enam fitur yaitu age, day, duration, campaign, pdays, dan previous. Data dibagi menjadi 36,168 baris training dan 9,043 baris testing. Hasil evaluasi menghasilkan R^2 sebesar 0.0087, MAE sebesar 1526.63, dan RMSE sebesar 3116.31. Nilai R^2 yang mendekati nol menunjukkan fitur yang digunakan hanya menjelaskan sebagian kecil variasi balance. MAE berarti rata-rata selisih absolut prediksi sekitar 1526.63 satuan balance, sedangkan RMSE yang lebih besar menunjukkan adanya beberapa kesalahan prediksi yang cukup besar

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum UTS yang telah dilakukan, dataset bank customers terdiri dari 45.211 baris dan 17 kolom. Hasil analisis menunjukkan tidak ada nilai yang hilang ataupun baris yang duplikat. EDA dilakukan dengan menganalisa sebaran target, histogram usia, kotak plot saldo, dan hubungan antar kolom numerik. Distribusi target terlihat tidak seimbang karena jumlah kelas no mencapai 39.922, sedangkan kelas yes hanya berjumlah 5.289. Pada proses klasifikasi, fitur kategorikal diproses menggunakan One-Hot Encoding dan fitur numerik dilanjutkan tanpa modifikasi. Data tersebut dibagi dengan proporsi 80:20 dan stratify=y. Model Decision Tree dengan max_depth=6 mencapai akurasi 90,11%, namun recall untuk kelas yes hanya 0,43 sehingga model ini belum optimal dalam mengidentifikasi kelas yes. Dalam percobaan regresi, model Regresi Linier menggunakan enam fitur numerik untuk memprediksi saldo yang menghasilkan R^2 sebesar 0,0087, MAE 1.526,63, serta RMSE 3.116,31. Hasil ini mengindikasikan bahwa model regresi pada pengaturan ini belum mampu menjelaskan variasi saldo dengan baik.

DAFTAR PUSTAKA

Adsavakulchai, S., & Chysirichote, T. (2025). A Machine Learning Approach to Predicting Bank Customer Behavior. Journal of Research and Applications in Science and Technology, 14(1). https://jraspublications.org/index.php/jras/article/view/157

Basysyar, F. M., Dikananda, A. R., & Kurnia, D. A. (2022). Prediction of Bank Customer Potential Using Creative Marketing Based on Exploratory Data Analysis and Decision Tree Algorithm. Ingénierie des Systèmes d'Information, 27(4), 597–604. https://doi.org/10.18280/isi.270409

Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. PeerJ Computer Science, 7, e623. https://doi.org/10.7717/peerj-cs.623

Esaki, T. (2021). Appropriate Evaluation Measurements for Regression Models. Chem-Bio Informatics Journal, 21, 59–69. https://doi.org/10.1273/cbij.21.59

Fernandes, A. A. A., Koehler, M., Konstantinou, N., Pankin, P., Paton, N. W., et al. (2023). Data Preparation: A Technological Perspective and Review. SN Computer Science, 4, 425. https://doi.org/10.1007/s42979-023-01828-8

Gustian, A. S., & Saeppani, A. (2026). Analisis Perbandingan Algoritma Regresi Linear dan Decision Tree untuk Prediksi Dropout Mahasiswa. *Merkurius*, *4*(1), 155–164. https://doi.org/10.61132/merkurius.v4i1.1362

Kholifah, B., Firmansyah, F.B., Sururi, N., & Nugraha, D.S. (2024). Exploratory Data Analysis dan Machine Learning dalam Memprediksi Employee Attrition. Nusantara Journal of Science and Technology, 1(2), hal. 38-47. https://ejurnal.unukase.ac.id/index.php/nujst/article/view/118

Mao, R., Shi, X., & Shi, Z. (2025). A Decision Tree Classification Algorithm Based on Two-Term RS-Entropy. Entropy, 27(10), 1069. https://doi.org/10.3390/e27101069